Information Theory for Data Science

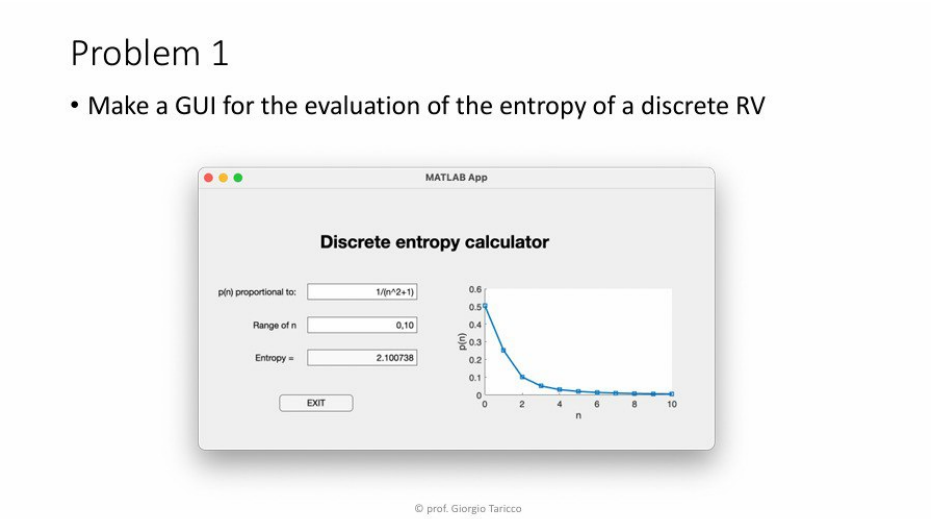
Master's Degree in Data Science and Engineering: Assignment 2

2024/10/DEC

Group Number: G 25

- Negin Azadeh Rezvani *s338144*
- Kamyar Fazlollah Nezad *s341418*
- Mohammadreza Karami *s342335*
- Fatemeh Saleh *s344410*
- Shahrzad Shivaei *s343616*

Problem 1 Part 1



Introduction

The aim of this report is to explain how to calculate entropy for a discrete random variable. To begin with, we start with a discrete probability distribution. The discrete random variable X has possible outcomes indexed by n. The probability $p(n)$ for each outcome is a function of n.

Probability Function

The probability mass function $p(n)$ describes the likelihood of each outcome n occurring. In this case, we provide a mathematical expression for $p(n)$. The probability for each n is given by:

$$p(n) = \frac{1}{n^2 + 1} \tag{1}$$

This defines the probability of each outcome.

If we consider n in the range $n_{\text{start}} = 0$ to $n_{\text{end}} = 10$, we can calculate $p(n)$ as follows:

$$\begin{aligned}
p(0) &= \frac{1}{0^2+1} = 1, \\
p(1) &= \frac{1}{1^2+1} = 0.5, \\
p(2) &= \frac{1}{2^2+1} = 0.2, \\
&\vdots \\
p(10) &= \frac{1}{10^2+1} = 0.0099.
\end{aligned}$$

The probability distribution must satisfy the requirement that the sum of all probabilities over the specified range is 1. To ensure this, we normalize the probabilities using the following formula:

$$p'(n) = \frac{p(n)}{\sum_{n=\text{start}}^{\text{end}} p(n)}. \tag{2}$$

For example:

$$\begin{aligned}
\sum_{n=\text{start}}^{\text{end}} p(n) &= 1 + 0.5 + 0.2 + \cdots + 0.0099 \approx 1.9818, \\
p'(0) &= \frac{1}{1.9818} \approx 0.5046, \\
p'(1) &= \frac{0.5}{1.9818} \approx 0.2523, \\
p'(2) &= \frac{0.2}{1.9818} \approx 0.1009, \\
&\vdots \\
p'(10) &= \frac{0.0099}{1.9818} \approx 0.0050.
\end{aligned}$$

Shannon Entropy

The entropy can be calculated using the following formula:

$$H = -\sum_n p'(n) \cdot \log_2 \big(p'(n)\big). \tag{3}$$

The Shannon entropy $H(X)$ quantifies the uncertainty or the amount of information contained in the random variable X.

The logarithmic term $\log_2\big(p'(n)\big)$ measures the information content of each outcome n. Higher values of $p(n)$ (i.e., more probable outcomes) result in lower contributions to the entropy, whereas lower values of $p(n)$ (i.e., less probable outcomes) contribute more to the entropy.

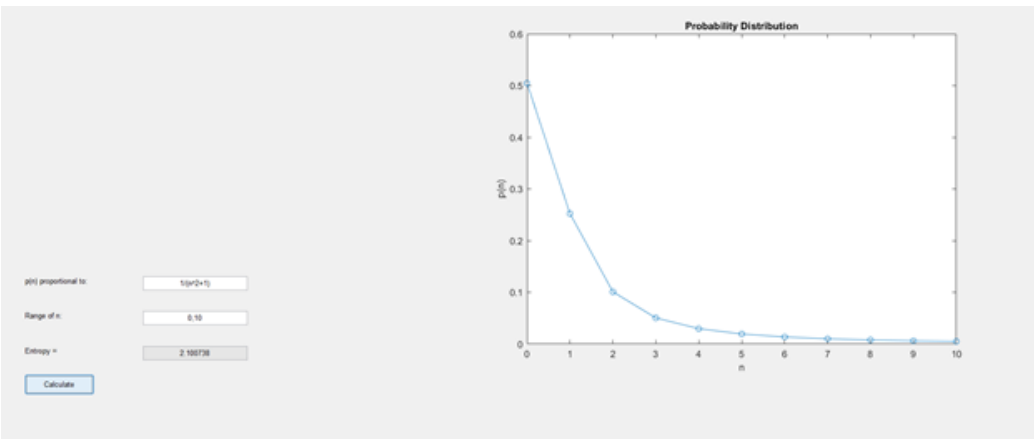
For example, calculating the entropy for the normalized probabilities:

$$\begin{aligned} \text{For } n = 0: \quad & -0.5046 \cdot \log_2(0.5046) \approx 0.4978, \\ \text{For } n = 1: \quad & -0.2523 \cdot \log_2(0.2523) \approx 0.5002, \\ & \text{For } n = 2 \text{ and further values, compute similarly.} \end{aligned}$$

Finally, summing all these terms:

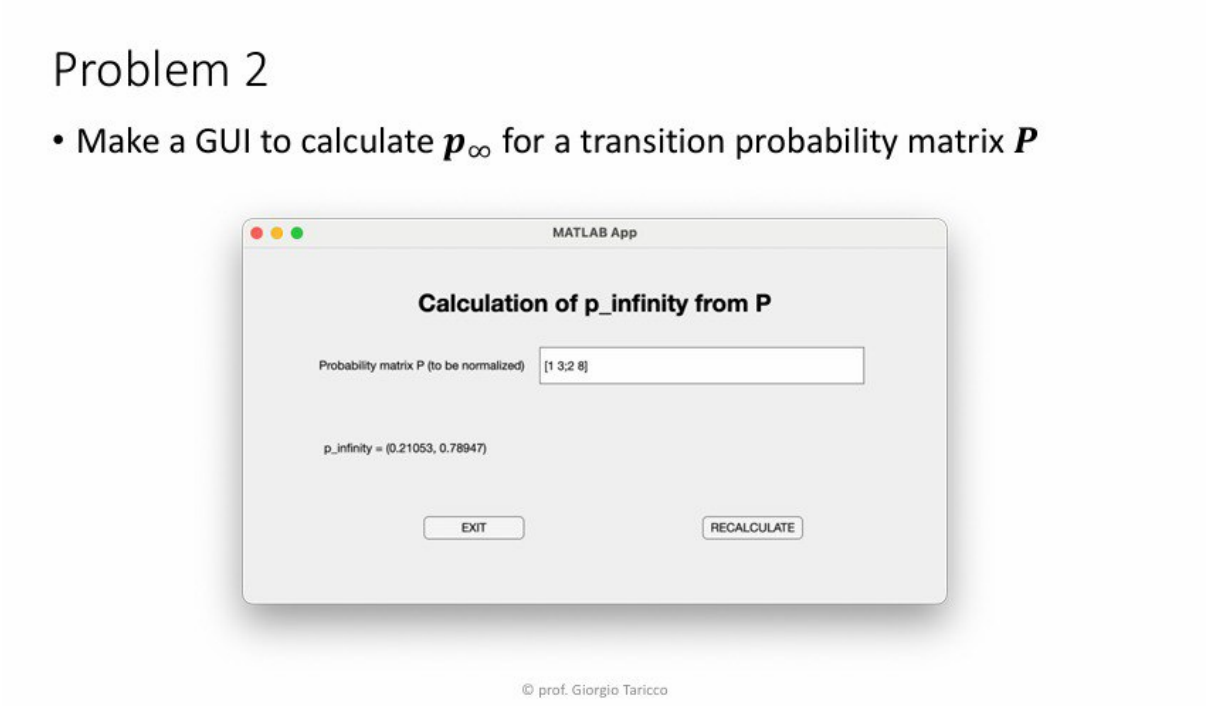
$$H \approx 2.10. \tag{4}$$

Conclusion



This is the GUI for the evaluation of the entropy of a discrete random variable. The GUI also provides a visualization of the probability distribution $p(n)$ by plotting the normalized probabilities against the values of n.

Problem 2 Part 1: Steady-State Probabilities in Markov Chains



Introduction

In this exercise, we focus on the idea of P_∞ (steady-state probabilities in a Markov chain) and how to compute it from a given probability matrix. Our goal is to determine the stationary state probability distribution, denoted as P_∞, which represents the long-term probabilities of being in each state of a system, assuming the probability distribution does not change over time.

Markov Chain

A Markov chain models systems that change over time by transitioning between different states. The key feature of a Markov chain is the *Markov property*, which means the future

state of the system depends only on its present state, not on the history of how it got there.

If there are n distinct states, the state space is defined as:

$$S = \{s_1, s_2, \ldots, s_n\}$$

where each s_i represents a specific state.

Transition Probability Matrix

The transition probability from one state to another in a system can be described by a matrix. Specifically, if the system starts in state s_i, the probability of reaching state s_j in one step is denoted by p_{ij}.

For longer transitions, the probability of moving from state s_i to state s_j in exactly k steps is given by the (i, j)-th entry of the matrix P^k. This matrix, denoted as P^k, is computed by multiplying the original transition matrix P by itself k times:

$$P^k = P \times P \times \cdots \times P \quad \text{(with } k \text{ multiplications)}$$

The transition matrix P satisfies the following properties:

1. P is a square matrix of size $n \times n$, where n is the number of states.
2. Each row of P sums to 1:

$$\sum_{j=1}^{n} p_{ij} = 1, \quad \text{for each row } i.$$

3. All entries p_{ij} are non-negative:

$$p_{ij} \geq 0, \quad \text{for all } i, j.$$

Steady-State Probabilities

To calculate the steady-state probabilities, we define $P_\infty = [p_1, p_2, \ldots, p_n]$, a row vector of size $1 \times n$, where p_i represents the long-term probability of being in state s_i. This vector satisfies the following equation:

$$P_\infty P = P_\infty$$

where:

- P_∞: A row vector representing the steady-state probabilities.
- P: The transition probability matrix.

The steady-state equation can be rewritten as a system of linear equations:

$$\sum_{j=1}^{n} p_i \cdot p_{ij} = p_i, \quad \text{for all } i = 1, 2, \ldots, n.$$

Normalization

To normalize the steady-state vector, calculate the sum of its components:

$$S = \sum_i P_\infty(i)$$

If $S = 1$, the vector satisfies the normalization condition. Otherwise, normalize the vector by dividing each component by S:

$$P^\infty_{\text{normalized}}(i) = \frac{P_\infty(i)}{S}.$$

After normalization, verify that the sum of the components is indeed 1:

$$\sum_{j=1}^{n} P_{\infty,j} = 1.$$

Solving the System

The steady-state probabilities can be found by solving the system of equations:

$$P_\infty(P - I) = 0$$

where I is the identity matrix of size $n \times n$.

To solve this, employ eigenvalue decomposition of the matrix $P^T P$ (the transpose of P multiplied by P):

$$\det(P^T P - \lambda I) = 0$$

where λ is the eigenvalue and I is the identity matrix.

The steady-state distribution corresponds to the eigenvector associated with the eigenvalue $\lambda = 1$. Normalize this eigenvector to ensure it represents valid probabilities:

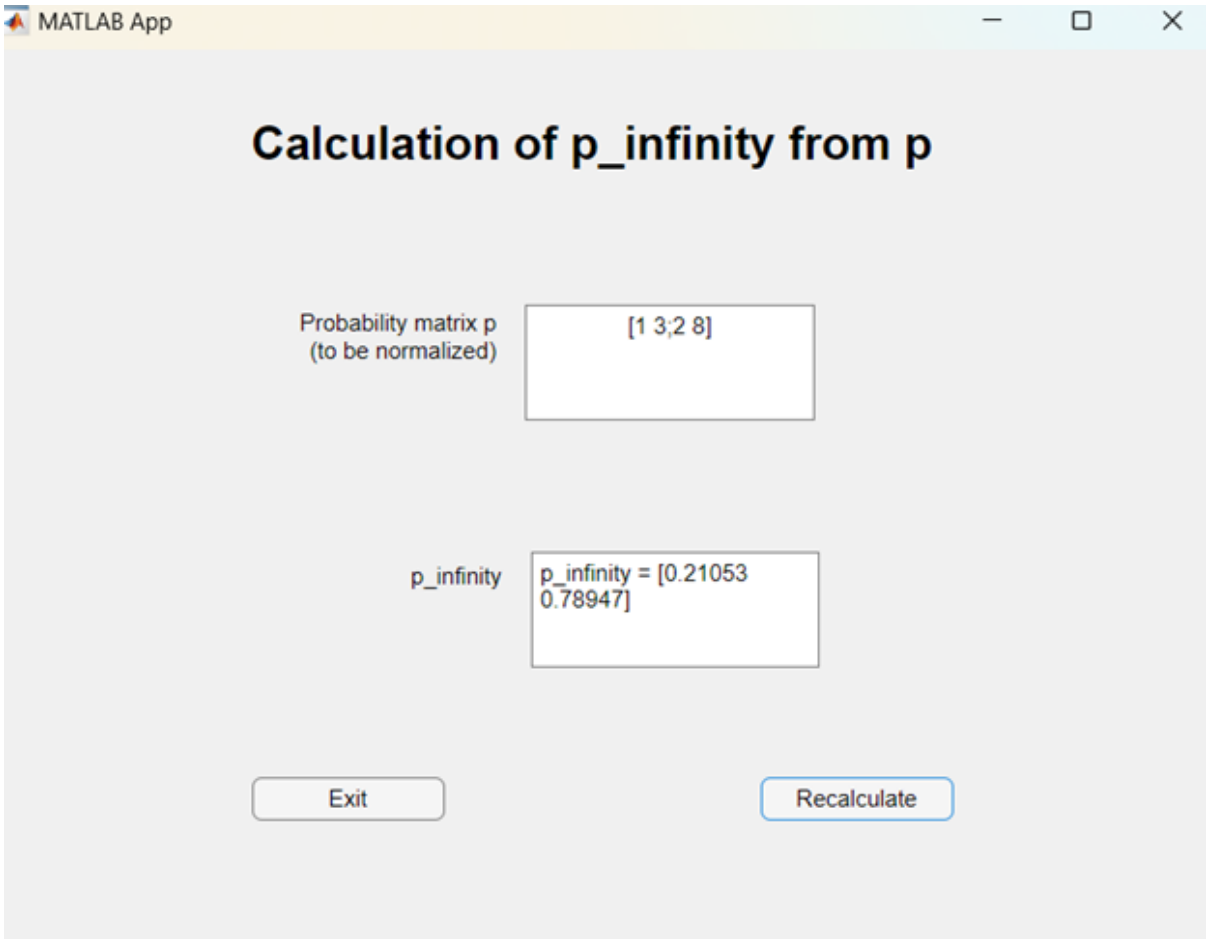
$$\sum_{j=1}^{n} P_{\infty,j} = 1.$$

Result

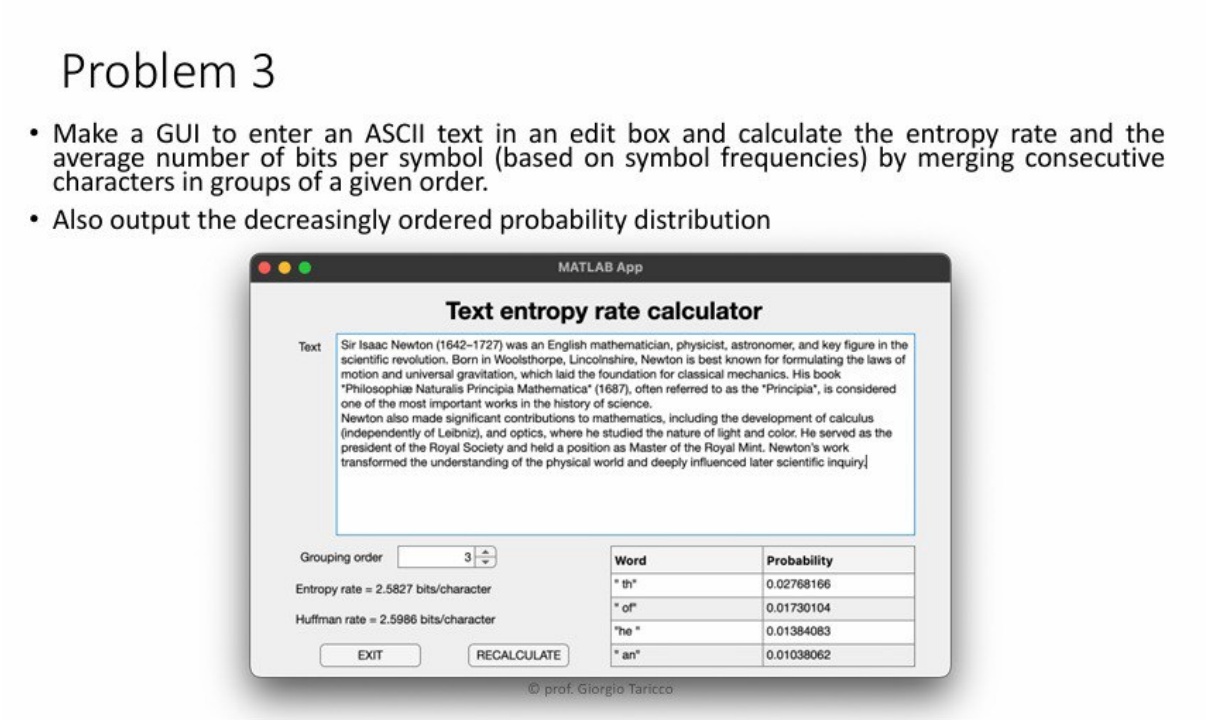
We created a GUI to calculate P_∞ for a given transition probability matrix. For example, inputting the matrix:

$$\begin{bmatrix} 1 & 3 \\ 2 & 8 \end{bmatrix}$$

allows the MATLAB app to compute the steady-state probabilities. The results are displayed in the accompanying figure.


MATLAB App
Calculation of p_infinity from p
Probability matrix p
(to be normalized)
[1 3;2 8]
p_infinity
p_infinity = [0.21053
0.78947]
Exit
Recalculate

Problem 3 part1: Entropy Rate and Average Bits per Symbol



Introduction

The goal of this task is to calculate the entropy rate and the average number of bits per symbol for a given string of ASCII text. The program analyzes the ASCII values of the characters and groups them based on a specified order size. It then calculates the probability of each symbol and sorts them by their probability.

Entropy Calculation

To calculate the entropy of a set of symbols, we use the following formula:

$$H(X) = -\sum_{i=1}^{N} p(x_i) \log_2 p(x_i)$$

In this case, the script calculates entropy for *n-grams* (sequences of n consecutive characters), where the symbols of the sequence are treated as unique entities. We can calculate the entropy of n-grams (where n is the grouping size) using the formula:

$$H(X_n) = -\sum_{i=1}^{M} p(x_i) \log_2 p(x_i)$$

where M is the number of unique n-grams in the text.

Average Bits per Symbol

By calculating the average number of bits per symbol, we determine how efficiently we can encode the data. This is given by the formula:

$$B_{\text{avg}} = \sum_{i=1}^{M} p(x_i) \log_2 \frac{1}{p(x_i)}$$

Grouping and ASCII Encoding

This part is key to how the program works. The text is transformed into numbers that we can analyze, which is done through ASCII encoding and then grouping characters into n-grams.

ASCII Encoding

The input text is converted into ASCII values. Each character is replaced by its corresponding ASCII number. This step allows us to analyze how often certain characters (or groups of characters) appear in the text, as numbers are easier to work with than letters.

Grouping into n-grams

Once the text is converted into ASCII values, the characters are grouped into chunks called *n-grams*. An n-gram is a sequence of n consecutive characters. The code groups the input text into n-grams based on the specified number n. After grouping, each n-gram is converted into a unique number by summing the ASCII values of its characters, with each value being weighted by powers of 256. This is because each ASCII character fits in a byte, and a byte can represent 256 different values.

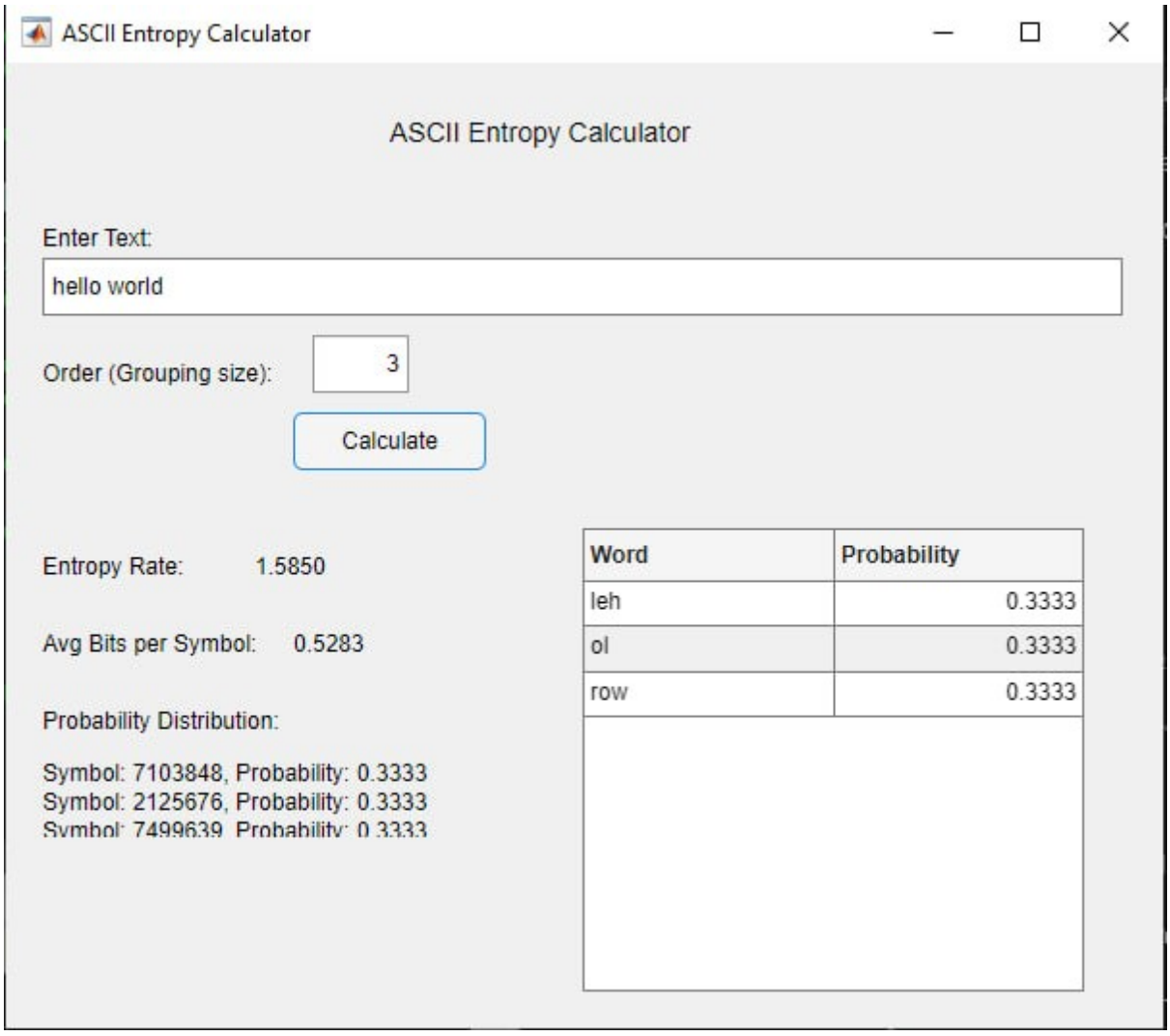
Calculating Probability Distribution

After creating the n-grams, the script calculates the frequency of each unique n-gram using the MATLAB function `unique()`. The result is an array of unique symbols (n-grams), and the counts of their occurrences are used to calculate their probabilities:

$$p_x = \frac{\text{Frequency of } x_i}{\text{Total number of n-grams}}$$

The n-grams are then sorted in decreasing order of probability, with the most frequent n-grams coming first. This sorting process is done using the MATLAB function `sort()`, which arranges the probabilities and n-grams in descending order.

Conclusion



The results are displayed in the GUI, with the probability distribution shown as text and also as a table in the MATLAB GUI, where each n-gram and its corresponding probability are listed.

Problem 4

- For each exercise provide the analytic solution (if it is possible to find an analytic solution, and include the parameter ranges)
- Otherwise, state that the analytic solution is impossible and provide the numerical solution (the correct choice is part of the evaluation)
- Calculate the entropy of the following distributions:
  1. $p(n) \propto \exp(-\lambda n)\,, n = 0,1,2, \dots$
  2. $p(n) \propto \exp(-n^2)\,, n = 0,1,2, \dots$
  3. $p(n) \propto n^{-4}, n = 1,2,3, \dots$
  4. $p(n) \propto \alpha^n, n = 1, \dots, N$
  5. $p(n) \propto (1 + n^2)^{-k}, n = 0,1,2, \dots$ for $k = 1,2$

HW2

Part A- Question 4

1.First we have to normalize the distribution.

$$Z = \sum_{n=0}^{\infty} \exp(-n) = \frac{1}{1 - \frac{1}{e}}.$$

The result is concluded from simple geometric series sum equation. Substituting this as p(x) in entropy formula, we'll have:

$$E = -\sum_{n=0}^{\infty} \frac{\exp(-n)}{Z} \ln\left(\frac{\exp(-n)}{Z}\right) = -\sum_{n=0}^{\infty} \frac{\exp(-n)}{Z} \left[-n - \ln(Z)\right] = \frac{1}{Z} \sum_{n=0}^{\infty} \exp(-n)n + \ln(Z).$$

Now we have to compute sum of $n * \exp(-n)$ from 0 to ∞ We know sum of x^n from 0 to ∞, and it is clear that we can just derivate it to reach the sum of the $n * x^n$ from 0 to ∞. Therefore, it is obvious that:

$$\sum_{n=0}^{\infty} n * x^n = \frac{x}{(1-x)^2}.$$

By substituting $\exp(-1)$ instead of x, we'll have:

$$\sum_{n=0}^{\infty} \exp(-n)n = \frac{\exp(-1)}{(1-\exp(-1))^2}.$$

So by substituting this and Z in the expression that we found for E, we'll have:

$$E = \frac{1}{Z}\sum_{n=0}^{\infty} \exp(-n)n + \ln(Z) = (1-\frac{1}{e}) * \frac{\exp(-1)}{(1-\exp(-1))^2} + ln(\frac{1}{1-\frac{1}{e}}) = \frac{\exp(-1)}{(1-\exp(-1))} + ln(\frac{1}{1-\frac{1}{e}})$$

2. We aim to calculate the entropy E of the distribution below, where Z is a normalizing factor:

$$p(n) = \frac{\exp(-n^2)}{Z}$$

but since the sum $\sum_{n=0}^{\infty} \exp(-n^2)$ does not have a closed form, we have to use numerical solutions.
Also, substituting the $p(n)$ given above in the entropy function, we will reach the expression below: (the last equality comes from the fact that the sum of the normalized distribution will be equal to 1)

$$S = -\sum_{n=0}^{\infty} \frac{\exp(-n^2)}{Z} \ln\left(\frac{\exp(-n^2)}{Z}\right) = -\sum_{n=0}^{\infty} \frac{\exp(-n^2)}{Z} \left[-n^2 - \ln(Z)\right]$$

$$= \frac{1}{Z} \sum_{n=0}^{\infty} \exp(-n^2) n^2 + \ln(Z).$$

To calculate this, we have to use numerical solutions. In order to do this, we used python code below:

And the results are:

$$Z \approx 1.386319, \quad S \approx 0.645665$$

```python
import math
s = 0
for n in range(0,100000):
  s += math.exp(-n**2)
E = 0
for n in range(0,100000):

  E += (math.exp(-n**2)*n*n)

print(s)
print(math.log(s)+(1/s)*(E))
```

```
1.3863186024133263
0.6456653446139602
```

Also I tried for 100 times more steps, but difference was less than 0.001.

3. First we try to normalize this distribution to have sum of 1. The sum in the following is known as the Riemann zeta function $\zeta(4)$.

$$Z = \sum_{n=0}^{\infty} n^{-4} = \zeta(4) = \frac{\pi^4}{90}.$$

Substituting , in the entropy function, we get:

$$E = -\sum_{n=1}^{\infty} \frac{n^{-4}}{Z} \ln\left(\frac{n^{-4}}{Z}\right) = -\sum_{n=1}^{\infty} \frac{n^{-4}}{Z} \left[-4\ln(n) - \ln(Z)\right] = \frac{4}{\frac{\pi^4}{90}} \sum_{n=1}^{\infty} n^{-4} \ln(n) + \frac{1}{\frac{\pi^4}{90}} \ln(\frac{\pi^4}{90})\zeta(4)$$

The first sum $\sum_{n=1}^{\infty} n^{-4} \ln(n)$ does not have a closed-form solution and requires numerical evaluation. Using a truncation at some large (e.g. 100) $n_{\max}$, we can compute this sum.
Thus, the entropy E can be approximated numerically by evaluating the sums. The Python code for calculating this value is as follows:

```
[19]
import math

s = 0
z = (math.pi**4)/90
for i in range(0,10000):
  s+= (n**(-4))*math.log(n)

print(4*s/z+math.log(z))
```

```
0.07910987306733964
```

4. The probability mass function $p(n)$ is proportional to a^n. To convert this into a valid probability distribution, we need to normalize it:

$$Z = \sum_{n=0}^{\infty} a^n = \frac{1}{1-a}$$

Hence, the normalized probability distribution is:

$$p(n) = \frac{a^n}{1-a}, \quad n = 0, 1, 2, \ldots$$

By substituting this in entropy function, we'll have:

$$E = -\sum_{n=0}^{\infty} \frac{a^n}{1-a} \log\left(\frac{a^n}{1-a}\right) = -\sum_{n=0}^{\infty} \frac{a^n}{1-a} \left(\log(a^n) - \log(1-a)\right) =$$

$$= -\sum_{n=0}^{\infty} \frac{a^n}{1-a} \left(n\log(a) - \log(1-a)\right) = -\sum_{n=0}^{\infty} \frac{a^n}{1-a} n\log(a) + \sum_{n=0}^{\infty} \frac{a^n}{1-a} \log(1-a)$$

It is obvious that the second term is equal to $log(1-a)$. The first term is equal to $alog(a)/(1-a)$ times the derivation of Z, so the result is equal to:

$$H(X) = -\frac{a}{(1-a)^3} \log(a) + \log(1-a)$$

5. To calculate the entropy, we first have to normalize the probability:

For $k = 1$ and $k = 2$, these sums do not have closed-form solutions, but are related to the Hurwitz zeta function $\zeta(k)$. Therefore, for numerical evaluation, we approximate these sums using truncation at large $N_{\max}$.

Thus, the normalized probability distribution is as follows:

$$p(n) = \frac{(1+n^2)^{-k}}{Z_k}, \quad k = 1, 2$$

We've calculated their approximate value with a python code (see below).

Substituting this probability in entropy function, we'll have:

$$E = -\sum_{n=0}^{\infty} \frac{(1+n^2)^{-k}}{Z_k} \log\left(\frac{(1+n^2)^{-k}}{Z_k}\right) = -\sum_{n=0}^{\infty} \frac{(1+n^2)^{-k}}{Z_k} \left(-k\log(1+n^2) - \log Z_k\right)$$

$$= k\sum_{n=0}^{\infty} \frac{(1+n^2)^{-k}}{Z_k} \log(1+n^2) + \log Z_k \sum_{n=0}^{\infty} \frac{(1+n^2)^{-k}}{Z_k}$$

The second sum is equal to 1, hence:

$$H(X) = k\sum_{n=0}^{\infty} \frac{(1+n^2)^{-k}}{Z_k} \log(1+n^2) + \log Z_k$$

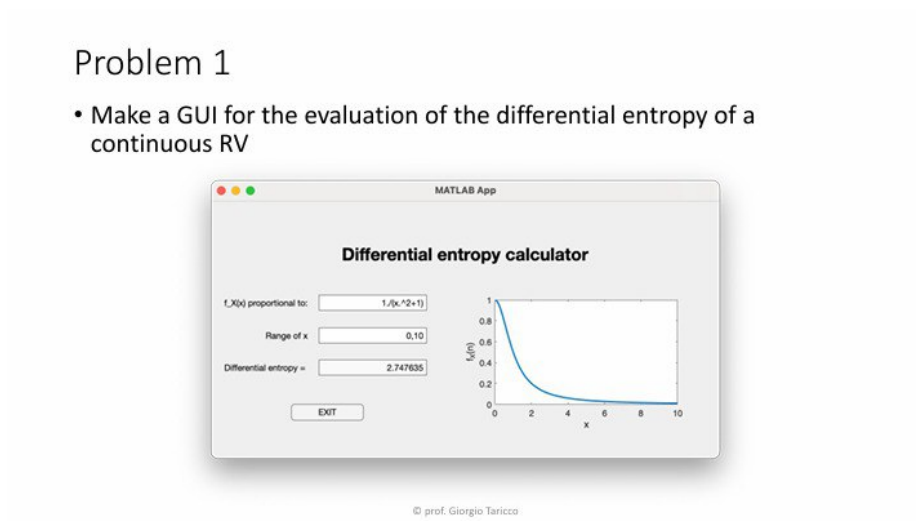
We cannot compute this analytically because we do not have closed-form solution of the first sum above for general k. These sums are related to special functions like the Hurwitz zeta function, and even though such functions exist, they do not yield simple closed-form expressions for entropy. Therefore, numerical methods are required to estimate the entropy. The calculated entropies are:

```python
import math
E1 = 0
E2 = 0
s1 = 0
s2 = 0
for n in range(0,1000000):
  s1 += (1+n**2)**(-1)
  s2 += (1+n**2)**(-2)

for n in range(0,1000000):
  n1 = ((1 + n**2)**(-1)) / s1
  n2 = ((1 + n**2)**(-2)) / s2
  E1 += n1*math.log(n1)
  E2 += n2*math.log(n2)
print('Z1 =', s1)
print('Z2 =', s2)
print('Entropy for k = 1 is', -E1)
print('Entropy for k = 2 is', -E2)
```

```
Z1 = 2.076673047468177
Z2 = 1.306836975422631
Entropy for k = 1 is 1.7742596267503163
Entropy for k = 2 is 0.7009278176290031
```

Problem1 Part 2: Differential Entropy Calculation for a Continuous Probability Distribution



According to the professor's notice, the output value of the differential entropy should be 2.046061, and the graph must . be normalized such that the y-axis value at x=0 equals 0.68

Introduction

In this task, the goal was to design and implement a MATLAB application to . First,ncalculate the differential entropy of a continuous probability distributio mawe define the probability function $f(x)$ and the range for x, and the progr computes the normalized probability density function (PDF), followed by ecalculating the differential entropy using the Shannon entropy formula. Th ed fortresult is displayed in the GUI, and a plot of the normalized PDF is genera . further analysis

Differential Entropy

The differential entropy $H(X)$ for a continuous random variable X with probability density function $f_X(x)$ is defined as:

$$H(X) = -\int_{-\infty}^{\infty} f_X(x) \log_2(f_X(x))\, dx$$

Normalization of the PDF

First, we need to normalize the probability distribution. This is done by computing the normalization constant Z and using the normalized PDF formula. The normalization constant is given by:

$$Z = \int_a^b f_X(x)\, dx$$

Once Z is computed, the normalized PDF is:

$$\tilde{f}_X(x) = \frac{f_X(x)}{Z}$$

Now, the differential entropy can be calculated by substituting the normalized PDF into the entropy formula:

$$H(X) = -\int_a^b \tilde{f}_X(x) \log_2(\tilde{f}_X(x))\, dx$$

Given Problem

For the given probability function:

$$f_X(x) = \frac{1}{x^2+1}$$

We begin by normalizing the function. We first compute the normalization constant Z:

$$Z = \int_0^{10} \frac{1}{x^2+1}\, dx$$

This is a standard integral, and its solution is:

$$\tan^{-1}(x)$$

Thus, the normalization constant Z is:

$$Z = \left[\tan^{-1}(x)\right]_0^{10} = \tan^{-1}(10) - \tan^{-1}(0) = \tan^{-1}(10) \approx 1.471$$

Now, the normalized PDF is:

$$\tilde{f}_X(x) = \frac{1}{(x^2+1)\cdot 1.471}$$

The differential entropy for the normalized PDF is given by:

$$H(X) = -\int_0^{10} \frac{1}{(x^2+1)\cdot 1.471} \log_2\left(\frac{1}{(x^2+1)\cdot 1.471}\right) dx$$

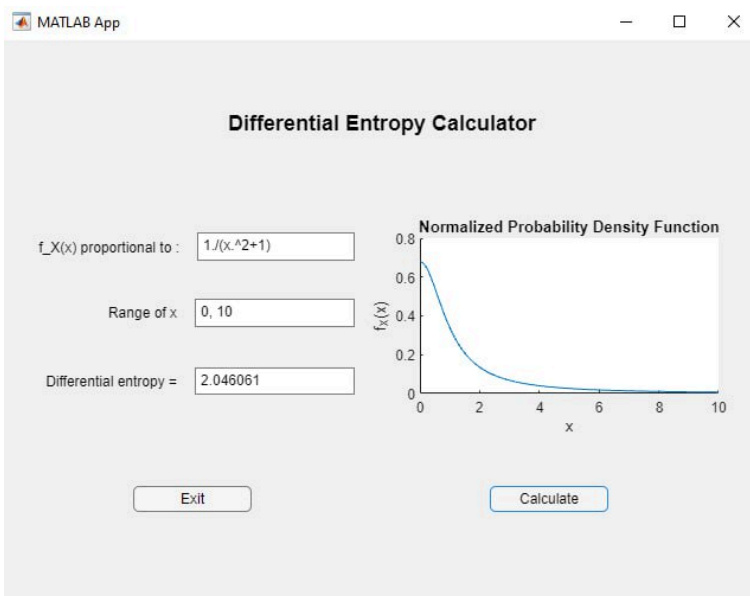
Breaking the logarithm into two parts:

$$\log_2\left(\frac{1}{(x^2+1)\cdot 1.471}\right) = \log_2(1) - \log_2\left((x^2+1)\cdot 1.471\right) = -\log_2\left((x^2+1)\cdot 1.471\right)$$

Therefore, the entropy integral becomes:

$$H(X) = \int_0^{10} \frac{\log\left((x^2+1)\cdot 1.471\right)}{(x^2+1)\cdot 1.471} dx$$

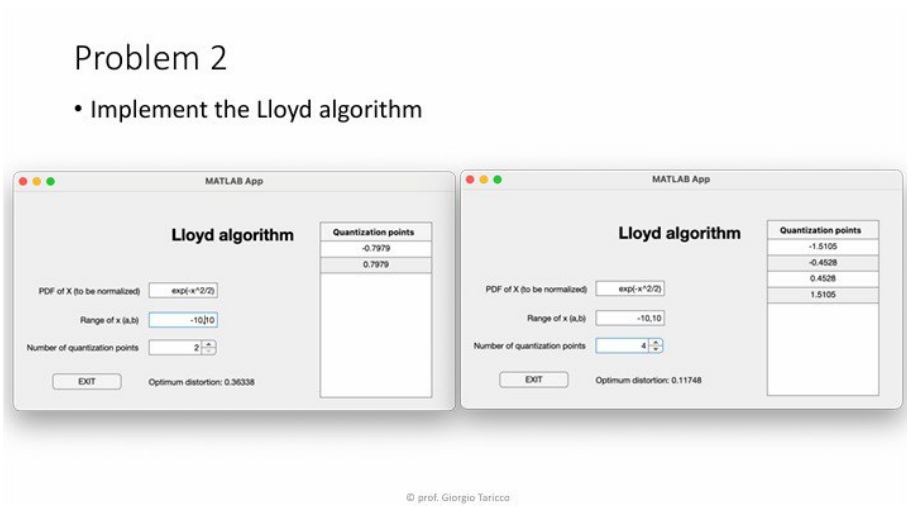
Result



Using the MATLAB GUI to compute the differential entropy, the result for the given probability function over the range [0, 10] is approximately:

$$H(X) \approx 2.046061$$

Problem 2 part 2: Quantization Using Lloyd Algorithm



Introduction

For the beginning, we have to initialize the range, number of quantization points, and the probability density function (PDF). Since the PDF is continuous, we can't directly compute it for each point; instead, we need to discretize the range $[a, b]$ into a set of points where we can evaluate the PDF numerically.

To do this, we divide the interval $[a, b]$ into N evenly spaced points. The formula used to calculate the positions of these points is:

$$X_i = a + i \cdot \frac{b-a}{N}, \quad i = 1, 2, \ldots, N$$

This discretization allows us to approximate the continuous PDF at these points. The choice of $N = 5000$ ensures that the points are densely packed, providing a good approximation of the continuous function, while still being computationally feasible for the algorithm to process efficiently.

Normalizing the PDF

The next step is to normalize the PDF. To do that, we require:

$$\int_a^b f_X(x)\,dx = 1$$

Numerically, the integral is computed using the trapezoidal rule:

$$\int_a^b f_X(x)\,dx \approx \sum_{i=1}^{N-1} \frac{(x_{i+1} - x_i)}{2} \left(f_X(x_i) + f_X(x_{i+1})\right)$$

This ensures that the PDF is properly scaled so that it represents a valid distribution, with the total probability summing to 1.

Lloyd Iteration

The main idea behind the algorithm is to repeatedly adjust the quantization points to reduce the difference between the original data and its simplified version. To do this, we first calculate the boundaries for defining the regions for each quantization point. These boundaries separate the data space into segments, each associated with a quantization point.

The boundaries between two neighboring quantization points, q_i and q_{i+1}, are calculated as follows:

$$b_i = \frac{q_i + q_{i+1}}{2}, \quad i = 1, 2, 3, \ldots, k-1$$

These boundaries split the range into k sections, with each quantization point q_i representing a particular section or region.

Next, we update the quantization points by calculating the average of the data points in each region. This update step reduces the difference between the original data and its quantized version in each region. Mathematically, the new quantization point q_i^{new} is calculated as:

$$q_i^{\text{new}} = \frac{\int_{b_i}^{b_{i+1}} x f_X(x)\,dx}{\int_{b_i}^{b_{i+1}} f_X(x)\,dx}$$

This update finds the center of the region, where the center is the point that is closest, on average, to all the other points in that region. It minimizes the overall distance between the center and the data points.

The algorithm continues running until the quantization points stop changing significantly (when the difference between the new and old points is small enough):

$$\max_i |q_i^{\text{new}} - q_i| > \text{tolerance threshold}$$

Distortion Calculation

Next, we calculate the distortion to find out how much error is introduced during the quantization process. The distortion for each quantization point is given by:

$$D_i = \int_{b_i}^{b_{i+1}} (x - q_i)^2 f_X(x)\, dx$$

Where $(x - q_i)^2$ is the squared error between the data point x and the quantization point q_i.

The total distortion D is the sum of the distortions across all regions:

$$D = \sum_{i=1}^{k} D_i$$

Where k is the number of quantization points. The main aim of calculating distortion is to know how well the data is being approximated by the quantization points.

Result

According to the question, we define the PDF as follows in the range $[-10, 10]$:

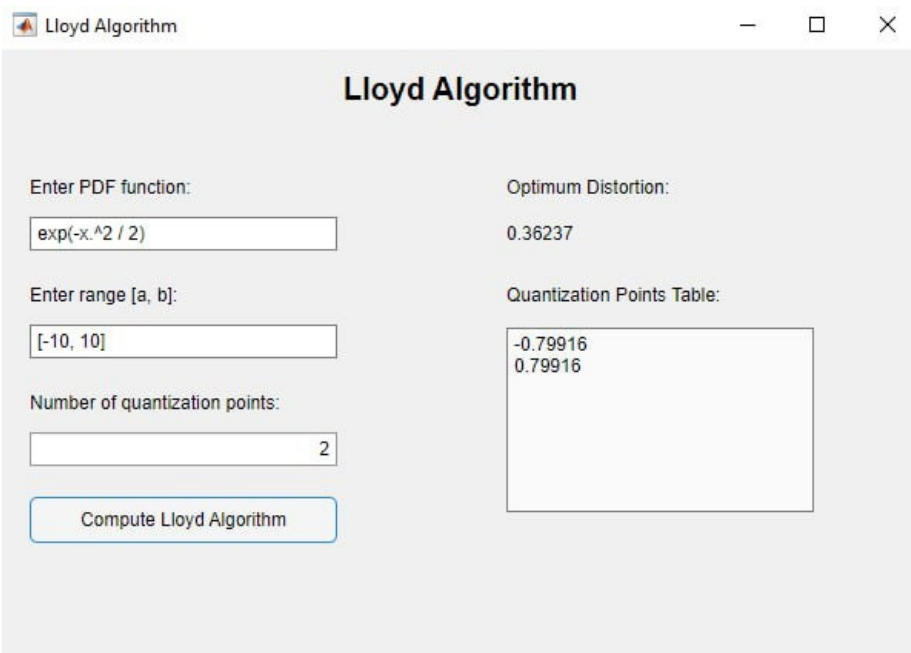
$$f_X(x) = \exp\left(-\frac{x^2}{2}\right)$$

By clicking on the compute button, we obtain the following result:

$$\text{Optimum Distortion} = 0.36237$$

And the quantization points table is:

$$q_1 = -0.79916, \quad q_2 = 0.79916$$


Lloyd Algorithm
Lloyd Algorithm
Enter PDF function:
exp(-x.^2 / 2)
Enter range [a, b]:
[-10, 10]
Number of quantization points:
2
Compute Lloyd Algorithm
Optimum Distortion:
0.36237
Quantization Points Table:
-0.79916
0.79916

Problem 3 part 2: LBG Algorithm for Quantizing a Two-Dimensional PDF

Problem 3

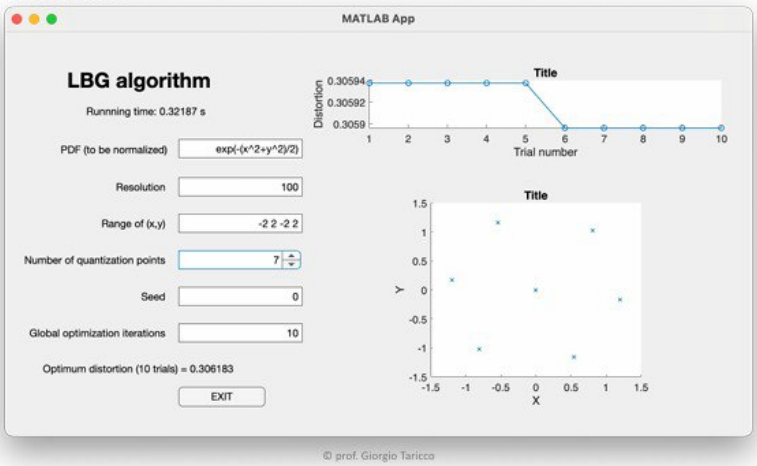
- Implement the LBG algorithm
  - Approximate the pdf by mass points on a square grid spanning a two-dimensional range with resolution N:

$$x_i = x_0 + \left(\frac{i}{N}\right)(x_N - x_0), y_j = y_0 + \left(\frac{j}{N}\right)(y_N - y_0), \qquad i, j \in \{0, \dots, N\}$$

  - Use several set of initial quantization points to approach global optimum
  - Plot minimum distortion vs trial number
  - Plot optimum quantization points after specified iterations

© prof. Giorgio Taricco

Problem 3



© prof. Giorgio Taricco

1 Introduction

In this report, we implement the LBG algorithm to quantize a two-dimensional probability density function (PDF). The PDF is approximated using mass points on a square grid, and we use the LBG algorithm to find the best quantization points that minimize the distortion.

The Linde-Buzo-Gray (LBG) algorithm is a vector quantization method used for data compression and representation. The main goal of the algorithm is to reduce the dimensionality of a dataset by finding a set of representative points that minimize the distortion between the original data points and their quantized versions.

The underlying PDF is given by:

$$P(X, Y) = e^{-\frac{X^2+Y^2}{2}} \tag{1}$$

Then, we normalize the PDF, ensuring that the total probability over the entire 2D space equals 1:

$$\int_{-\infty}^{\infty} \int_{-\infty}^{\infty} P(x, y)\, dx\, dy = 1 \tag{2}$$

2 The LBG Algorithm

We create a grid of points in the two-dimensional space x and y using the following formulas:

$$X_i = \texttt{linspace}(\texttt{minX}, \texttt{maxX}, \texttt{resolution}) \tag{3}$$

$$Y_i = \texttt{linspace}(\texttt{minY}, \texttt{maxY}, \texttt{resolution}) \tag{4}$$

Let the 2D space be represented by $\mathbb{R}^2$, and define the grid points on a square grid:

$$G = \{(x_i, y_i)\}_{i,j} \quad \text{where} \quad i, j \in \{1, 2, \ldots, N\} \tag{5}$$

Here, x_i and y_i represent the coordinates of the grid points in x and y, respectively.

In vector quantization, the goal is to partition the space into regions and assign a codeword to each region. Let the number of codewords be M, where M is the number of quantization points. Initially, we select M random points from the grid:

$$\mathbf{c}_k^{(0)} \quad \text{for} \quad k = 1, 2, \ldots, M \tag{6}$$

These initial centroids $\mathbf{c}_k^{(0)} = (x_k^{(0)}, y_k^{(0)})$ are chosen randomly. This forms a 2D grid of X and Y points using the `meshgrid` function:

$$X, Y \in \mathbb{R}^{\text{resolution} \times \text{resolution}} \tag{7}$$

Each $(X_{i,j}, Y_{i,j})$ corresponds to a grid point. The PDF expression is evaluated over the entire grid, where each point in the grid has a corresponding PDF value $Z_{i,j}$:

$$Z_{i,j} = e^{-\frac{x_i^2+y_i^2}{2}} \tag{8}$$

After evaluation, the PDF is normalized:

$$Z_{i,j} \to \frac{Z_{i,j}}{\sum_{i,j} Z_{i,j}} \tag{9}$$

The normalization ensures that the sum of all PDF values equals 1.

3 Calculating the Distortion

The next step is calculating the distortion to assess how well the quantization approximates the original continuous PDF. The distortion is computed using the formula:

$$D(t) = \sum_{i=1}^{N} \sum_{j=1}^{N} Z(x_i, y_i) \min \left[(x_i - q_{k,x})^2 + (y_i - q_{k,y})^2 \right] \tag{10}$$

This measures the weighted squared distance between each grid point and its nearest quantization point, where the weight is the probability mass at each grid point.

4 Distance Calculation

The distance between a grid point $\mathbf{g} = (g_x, g_y)$ and a centroid $\mathbf{c}_k = (c_{kx}, c_{ky})$ is computed using the Euclidean distance:

$$d(\mathbf{g}, \mathbf{c}_k) = \sqrt{(g_x - c_{kx})^2 + (g_y - c_{ky})^2} \tag{11}$$

For each grid point $\mathbf{g}$, assign it to the centroid $\mathbf{c}_k$ such that:

$$k^* = \arg\min d(\mathbf{g}, \mathbf{c}_k) \tag{12}$$

Once the grid points are assigned to the nearest centroids, the centroids are updated by computing the weighted average of the grid points assigned to each centroid:

$$q_{k,x}^{(t+1)} = \frac{\sum_{(x_i,y_i)\in C_k} x_i Z(x_i, y_i)}{\sum_{(x_i,y_i)\in C_k} Z(x_i, y_i)} \tag{13}$$

$$q_{k,y}^{(t+1)} = \frac{\sum_{(x_i,y_i)\in C_k} y_i Z(x_i, y_i)}{\sum_{(x_i,y_i)\in C_k} Z(x_i, y_i)} \tag{14}$$

The updated position is the weighted centroid of all grid points assigned to each quantization point.

5 Multiple Initializations for Global Optimum

To ensure the algorithm finds the best possible solution and avoids suboptimal results, it is run multiple times with different starting points for the quantization points. At each step, the algorithm tracks the distortion, and the set of quantization points that results in the lowest distortion is chosen as the final solution.

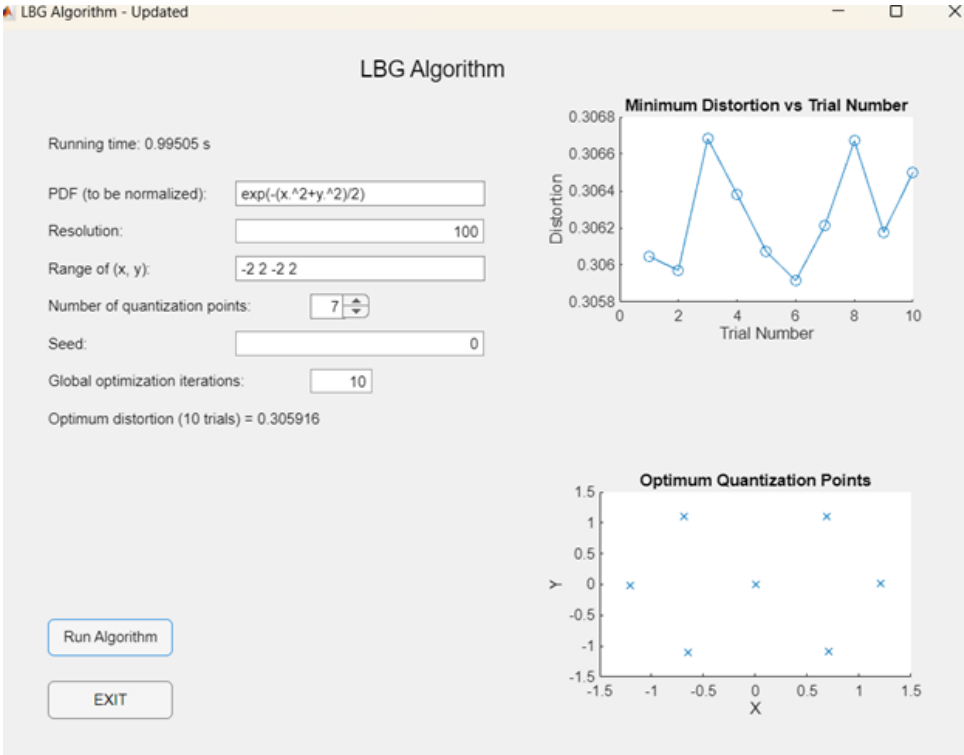
6 Stopping Criterion

The algorithm stops when one of the following conditions is met:

1. A preset number of iterations T is reached.
2. The distortion function stabilizes, i.e., the difference in distortion between two consecutive iterations becomes smaller than a specified threshold ϵ:

$$|D(t+1) - D(t)| < \epsilon \tag{15}$$

7 Results



The PDF is defined as:

$$P(X, Y) = e^{-\frac{X^2+Y^2}{2}} \tag{16}$$

After running the LBG algorithm using the MATLAB GUI, the optimum distortion obtained is 0.305916.

7.1 Minimum Distortion vs Trial Number

The first plot demonstrates the variation in distortion values across 10 trials. The distortion values fluctuate between approximately 0.3058 and 0.3068, indicating consistent but minor variations.

7.2 Optimum Quantization Points

The second plot shows the spatial distribution of quantization points in a two-dimensional plane. These points are symmetrically distributed within the range of $[-1.5, 1.5]$ on both axes, ensuring a balanced quantization scheme.

Problem 4

- Calculate the differential entropy of the following pdfs:
  1. $f_X(x) = \frac{u(x-a)-u(x-b)}{b-a}$
  2. $f_X(x) = \lambda e^{-\lambda x} u(x)$
  3. $f_X(x) = \frac{1}{\Gamma(n)} x^{n-1} e^{-x} u(x)$

HW2

Part B - Question 4

1. This PDF represents a uniform distribution between a and b. Specifically, the PDF is: (Note that the u(x) is a heaviside function)

$$f(x) = \begin{cases} \frac{1}{b-a} & \text{if } a \leq x < b, \\ 0 & \text{otherwise.} \end{cases}$$

Since $f(x) = \frac{1}{b-a}$ for $x \in [a, b]$ and $f(x) = 0$ otherwise, we only need to compute the integral over the interval $[a, b]$:

$$H(f) = -\int_a^b \frac{1}{b-a} \log\left(\frac{1}{b-a}\right) dx.$$

Since $\frac{1}{b-a}$ and the logarithmic term is constant over the interval, we can pull them out of the integral:

$$H(f) = -\frac{1}{b-a} \log\left(\frac{1}{b-a}\right) \int_a^b 1\, dx - = \frac{1}{b-a} \cdot \log\left(\frac{1}{b-a}\right) \cdot (b-a) = -\log\left(\frac{1}{b-a}\right).$$

Thus, the differential entropy is as follows:

$$H(f) = \log(b-a).$$

2. The entropy can be written as:

$$H(f) = -\int_{-\infty}^{\infty} \lambda e^{-\lambda x} \log(\lambda e^{-\lambda x}) u(x)\, dx = -\int_{0}^{\infty} \lambda e^{-\lambda x} \log(\lambda e^{-\lambda x})\, dx$$

By expanding the logarithmic term the entropy expression becomes:

$$H(f) = -\int_{0}^{\infty} \lambda e^{-\lambda x} \left(\log(\lambda) - \lambda x\right) dx$$

We can now split the integral, wee know it is sum over exponential distribution with λ parameter, hence it is eqal to 1:

$$H(f) = -\log(\lambda) \int_{0}^{\infty} \lambda e^{-\lambda x}\, dx + \lambda \int_{0}^{\infty} \lambda x e^{-\lambda x}\, dx = -\log(\lambda) + \lambda \int_{0}^{\infty} \lambda x e^{-\lambda x}\, dx$$

We can compute the integral $\int_0^\infty \lambda x e^{-\lambda x}\, dx$ using integration by parts. Let $u = x$, $dv = \lambda e^{-\lambda x} dx$, so that $du = dx$, and $v = -e^{-\lambda x}$. We get:

$$\int_{0}^{\infty} \lambda x e^{-\lambda x}\, dx = \left[-x e^{-\lambda x}\right]_0^\infty + \int_{0}^{\infty} e^{-\lambda x}\, dx = 0 + \frac{1}{\lambda}$$

Thus, the second integral is:

$$\int_{0}^{\infty} \lambda x e^{-\lambda x}\, dx = \frac{1}{\lambda}$$

Substituting both integrals back into the expression for entropy, we get the final value for the entropy:

$$H(f) = -\log(\lambda) + 1$$

3. Since $u(x)$ implies $f(x) = 0$ for $x < 0$, we integrate from 0 to ∞. Also by expanding the logarithm of $f_X(x)$ and substituting in entropy formula, we'll have:

$$H(f) = -\int_0^\infty \frac{1}{\Gamma(n)} x^{n-1} e^{-x} \left[-\log\Gamma(n) + (n-1)\log x - x\right] dx$$

Separating into terms:

$$H(f) = \frac{\log\Gamma(n)}{\Gamma(n)} \int_0^\infty x^{n-1} e^{-x} dx - \frac{n-1}{\Gamma(n)} \int_0^\infty x^{n-1} e^{-x} \log x \, dx + \frac{1}{\Gamma(n)} \int_0^\infty x^n e^{-x} dx$$

We should keep in mind that the $\Gamma(n)$ is independent of x and can be taken out of the integral. Evaluating each term:

$$\int_0^\infty x^{n-1} e^{-x} dx = \Gamma(n)$$

$$\int_0^\infty x^{n-1} e^{-x} \log x \, dx = \Gamma(n)\psi(n) \quad \text{(where } \psi(n) \text{ is the digamma function)}$$

$$\int_0^\infty x^n e^{-x} dx = \Gamma(n+1) = n\Gamma(n)$$

Substituting these results:

$$H(f) = \log\Gamma(n) - (n-1)\psi(n) + n$$

Thus, the entropy of the Gamma distribution is:

$$H(f) = \log\Gamma(n) - (n-1)\psi(n) + n$$

Here, you can see calculations for both Numerical and Analytical entropies:

```python
import numpy as np
from scipy.special import gamma, digamma, gammaln
from scipy.integrate import quad

def gamma_entropy(n):
    term1 = gammaln(n)
    term2 = -(n - 1) * digamma(n)
    term3 = n
    entropy = term1 + term2 + term3
    return entropy

def f(x, n):
    return (1 / gamma(n)) * (x ** (n - 1)) * np.exp(-x)

def log_f(x, n):
    return -np.log(gamma(n)) + (n - 1) * np.log(x) - x

def numerical_integrals(n):
    entropy_integral, _ = quad(lambda x: -f(x,n) * log_f(x,n), 0, np.inf)

    return entropy_integral
#Example n:
n = 2

entropy_analytical = gamma_entropy(n)
print("Analytical entropy:", entropy_analytical)

entropy_numerical = numerical_integrals(n)
print("Numerical entropy:", entropy_numerical)

```

```
Analytical entropy: 1.5772156649015328
Numerical entropy: 1.5772156649015312
```